

Distributed Systems - 15_12

Today we'll build the **Byzantine Reliable Broadcast**.

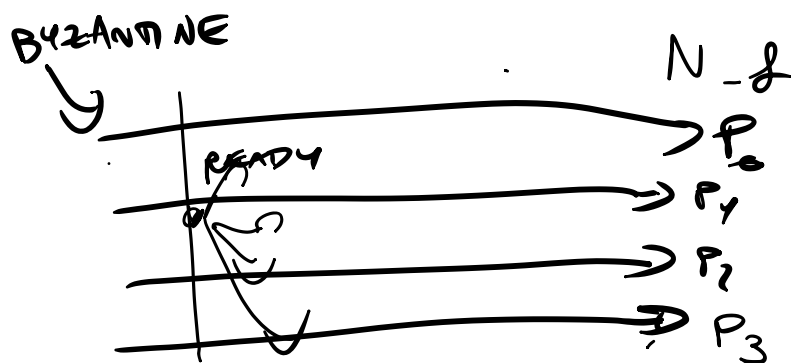
It always has a single sender process (for each instance). *If we want p_1 to send a message, we need an instance for p_1 and for specific message m_1 .*

The properties are the same as the BCB, plus a new one:

- (Totality): If some message is delivered by any correct process, then every correct process eventually deliver that message.

Our job now is to add to the previous algorithm a new phase, that makes them behave in a way that is total.

How can we make our correct processes agree on whether they have to deliver a message? We could make the correct process send a message to all the others, telling them he is ready to deliver the message.



Now, is one ready message enough? No, it could have been sent by the byzantine process. So, we have to decide the threshold of "ready" messages that we need in order to deliver a message.

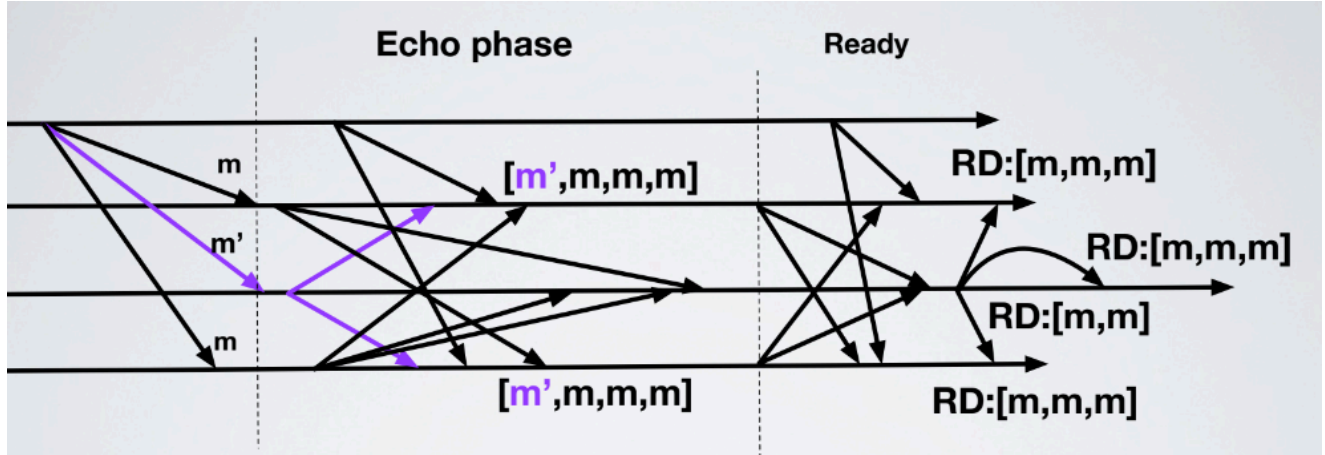
How many corrects do we have in the system? $N - f$. We'll have to deliver a message only when we see $N - f$ of "ready" messages (or less..., we can't for sure have more of that value).

This algorithm has now a "two-threshold" behaviour.

Now, if I'm correct, and see that another process is sending a ready message, then by knowing the sender is correct we can be sure that the message is ok, and we can amplify that. The problem is that we don't know if the sender is correct or byzantine. There is however a specific case in which I know it, and it's when we see $f + 1$ ready messages for a certain message (as one of them will be for sure coming from a correct process).

So, we'll have two thresholds:

- One for $f + 1$
- One for $N - f$, when we see a message replicated $N - f$ times, we deliver a message to the upper layer.



What can we see? In the first phase, processes echoes the message they've received, and we see that everyone reaches the threshold k except for p_3 . Then the ready phase starts, and processes who've reached the byzantine quorum starts sending the ready messages. We'll have that p_2 and p_4 will reach the $N - f$ quorum, thus delivering the message. p_3 , however, still did not deliver the message, but it has reached a threshold of $f + 1$, thus he can start amplifying the ready, receiving himself his ready message, and thus reaching $N - f$.

Let's now see the code:

Algorithm 3.18: Authenticated Double-Echo Broadcast

Implements:

ByzantineReliableBroadcast, **instance** *brb*, with sender *s*.

Uses:

AuthPerfectPointToPointLinks, **instance** *al*.

upon event $\langle \text{brb}, \text{Init} \rangle$ **do**

sentecho := FALSE;
sentready := FALSE;
delivered := FALSE;
echos := $[\perp]^N$;
readys := $[\perp]^N$;

upon event $\langle \text{brb}, \text{Broadcast} \mid m \rangle$ **do**

forall $q \in \Pi$ **do**
 trigger $\langle \text{al}, \text{Send} \mid q, [\text{SEND}, m] \rangle$;

upon event $\langle \text{al}, \text{Deliver} \mid p, [\text{SEND}, m] \rangle$ **such that** $p = s$ **and** *sentecho* = FALSE **do**

sentecho := TRUE;
forall $q \in \Pi$ **do**
 trigger $\langle \text{al}, \text{Send} \mid q, [\text{ECHO}, m] \rangle$;

upon event $\langle \text{al}, \text{Deliver} \mid p, [\text{ECHO}, m] \rangle$ **do**

if *echos*[*p*] = $\perp$ **then**
 echos[*p*] := *m*;

upon exists $m \neq \perp$ such that $\#(\{p \in \Pi \mid \text{echos}[p] = m\}) > \frac{N+f}{2}$

and *sentready* = FALSE **do**

sentready := TRUE;

forall $q \in \Pi$ **do**

trigger $\langle \text{al}, \text{Send} \mid q, [\text{READY}, m] \rangle$;

upon event $\langle \text{al}, \text{Deliver} \mid p, [\text{READY}, m] \rangle$ **do**

if *readys*[*p*] = $\perp$ **then**

readys[*p*] := *m*;

upon exists $m \neq \perp$ such that $\#(\{p \in \Pi \mid \text{readys}[p] = m\}) > f$

and *sentready* = FALSE **do**

sentready := TRUE;

forall $q \in \Pi$ **do**

trigger $\langle \text{al}, \text{Send} \mid q, [\text{READY}, m] \rangle$;

upon exists $m \neq \perp$ such that $\#(\{p \in \Pi \mid \text{readys}[p] = m\}) > 2f$

and *delivered* = FALSE **do**

delivered := TRUE;

trigger $\langle \text{brb}, \text{Deliver} \mid s, m \rangle$;

Now, the left part is basically the algorithm seen for Byzantine Consistent

Broadcast.

In the init, we have a `sentready` boolean value, stating if we sent the ready message. We also have the `readys`, an array in which we memorise the message for the process who sent the ready message.

Now, in the first handler in the right part, instead of delivering, I send the ready message to everyone.

In the third right handler, we check if we didn't send the ready message yet, and if we didn't we set `sentready` to `True`, and we have $f + 1$ ready messages for m , we amplify the ready.

In the last handler (checking $> 2f$ is equivalent to checking $> N - f$), we check if we have $2f + 1$ ready messages for a message m , and in that case we deliver.

Let's now take a look at the proofs:

Validity: if the sender is correct at least $n-f$ processes do the echo (all the correct) of m . For our discussion of the consistent byz. Cast, we know that all corrects will send ready messages. Thus each correct receives (at least) $2f+1$ ready for m and delivers.

No duplication: Immediate from the auth. channel

Integrity: Immediate from the auth. channel

Now, let's see consistency. Let's say there are two correct processes, delivering messages for m and m (two different coloured messages). If this happens, then it means that a process reaches the $2f + 1$ threshold for m , and another process reaches this threshold for m . To reach this threshold, we need however one correct process sending ready for m , and one sending ready for m . Is it possible that there are two corrects are sending ready for the different m and m ? No, for the byzantine quorums, otherwise the previous algorithm would be incorrect.

Another possibility is that there is sending the ready in the first right handler, and one in the third right handler. We also have to show this is impossible. If someone sends a ready in the third right handler, then it has seen $f + 1$ ready messages for m .

I got lost after this...

Let's now see totality property. Let us suppose there is correct process p delivering m . Then, it executed the last trigger. If p delivers m , then it has seen a number of echo $= 2f + 1$. Of these $2f + 1$, at least we have $f + 1$ correct processes. These $f + 1$ are sending the ready messages in one of the points in the handlers. Those sending are broadcasts, so they are sending the ready to everyone in the system. If we have $f + 1$ processes sending ready to everyone, then we'll have every correct

executing the amplification handler (as everyone has reached the $f + 1$ threshold). Now we'll have that a certain point every correct is amplified, and we'll have everyone reaching the $N - f$ threshold.

What is the message complexity? We'll have N messages for the broadcast, then N^2 for the echo. For the first right, we have N , and every correct process either executes this handler, or the third right, having N messages. We still have $O(N^2)$. The message delay? It's 3, as it's a three-step algorithm, one for the sending, one for the echo, and one for the ready phase.

What is the resiliency? It's $f < \frac{1}{3}$, which means $N \geq 3f + 1$.

Until known we are using MACs to prevent the byzantine sending a message claiming to be another guy. If we use signatures, we could have a resiliency of $f < N$.

We'll use a synchronous system.

The first observation: if the system is synchronous, and I expect a message to arrive, and it does not arrive, then it means that the sender did not send the message (it's not possible that it is a delay).

Second observation: in the byzantine consistent broadcast, we have an original sender sending a message to everyone. Suppose we have two messages m and m' signed by the sender. Since signatures cannot be forged, if I see two signatures, then it means that the sender has created a signature for m and m' . Since with this kind of broadcast we can send only one message, if we see two different messages signed by the sender, then it means the sender is byzantine.

Third observation: If someone has a proof that someone is byzantine, he can show this proof to everyone in one round, every correct will see this proof and trust it.

Let's take an example. We have p_1 sender and three more processes. In the first round p_1 will send m to everyone, as well as the signature σ (so $[m, \sigma]$). If the sender is correct, it sends a single message and a single signature. When a process sees at the end of r_1 , what it should do? Can it deliver immediately? No, as if the sender is not correct, two different processes could receive m and m.

Unfortunately we don't know yet if the sender is correct or not. Before doing an echo, we check if the signature σ is valid for m . Only then, I send $[m, \sigma]$ to everyone. At the end of the round, all the corrects, if the sender is byzantine, will see two different messages with two valid signatures, and this would lead to the

corrects rejecting the messages.

Consistency:

Suppose two correct processes p and q deliver two different messages m and m' . If p delivers m then it means that it has seen at round 1 a signed SEND with m . At round 2 it sends this message to q . At round 2 q receives the ECHO from p , this ECHO contains a signed message SEND with payload m , that prevents q to deliver m' . This contradiction ends the proof.

Fact 1: if the sender is correct, no one can forge its signature. So it is not possible to create a valid SEND that contains a message different from the one sent.

Validity: The correct sender p will send the same SEND message m to all processes in round 1.

At round 2 the only valid ECHO are the ones that contain m . Thus no correct process can receive a valid ECHO that prevents her/him to deliver m .

Byzantine agreement

The byzantine agreement will be seen in the synchronous model.

The property that we have to change the definition is not only in the agreement (of which we have to have the non-uniform one), but also validity (which was that "if a process decides v , then v was proposed by someone").

We don't fully like this one, rather we'll see this one:

- (Correct-Input validity): if a process decides v , then v was proposed by some **correct** process.

It has some problems, like how to implement it in some situations (in some it's impossible).

For this, another property was proposed:

- (All-same validity): If all correct processes start with value v , then the decision value must be v .

This property has a lot of applications in real systems, like in distributed sensors. We'll have that the value decided is the one proposed by correct processes.

For the byzantine agreement we'll see two things:

- Proof that shows the impossibility when the number of processes is $N = 3f$, so we have that $f < \frac{N}{3}$.
- The king algorithm that solves BA as long as $N = 3f + 1$.

We assume authenticated channels (but no signatures).

Let's take a look at the proof.

Initially the proof is only for a system of 3 processes, showing we cannot tolerate $f = 1$ failures. The proof is by contradiction. Let us assume that an algorithm exists, and it tolerates $f = 1$ failures in a system of three processes. We'll show that this algorithm will be wrong in some situations.

The proof uses "scenario argument", a technique.

The scenario technique is to take the algorithm for three processes, and run it on a bigger system. If I run it on a system of 6 processes, the algorithm is not correct anymore. However, we could run a set of rules still on a system of 6, and those rules would produce an action (something will happen anyways). All processes are correct in this system of 6 processes. The processes in grey will propose 0 in the consensus, the ones in blue will propose 1 (for diagram check slides). Because we're running the algorithm on more processes than is intended to, it's possible that the algorithm never terminates (termination is not granted anymore).

Let's show the algorithm terminates anyways. Let's take p_2 first and say that it never terminates. If it never terminates, then I could take the $p_0 - p_2$ and $p_3 - p_1$ (p_1 blue) link and check the messages that are exchanged.

Then I can create a system of two processes (grey p_2 and p_3), and send messages in these links, such that p_2 and p_3 hangs forever. Now, this implies I can do this with a system of three processes, having a new process that is Byzantine. We would have a new ring, in which p_2 still does not terminate. This is however a contradiction, as the algorithm is correct, tolerating one byzantine, and has to work. This means that, even in the ring of 6, I have that all processes have to terminate (or we could recreate the scenario shown above for every possible couple).

Now, what has to be the output of p_2 in the system? It could either be 0 or 1 (values proposed in the system). However, the system (with yellow cut) behaves like the ring with Byzantine process. On that ring, what has to be the output of p_2 ? Remember that p_2 and p_3 proposed zero, so p_2 has to decide 0 for all-same validity. This implies that even in the ring of 6, p_2 has to decide 0. This applies also to p_3 , but also p_1 (doing the same technique with cutting the links).

Let us examine the guys in blue. Let us examine p_1 . It has to terminate (for the same argument seen before). Also, we'll have blue p_1, p_2, p_3 decide 1.

Now, what happens if we do a cut including a grey and blue process? We would have that the grey process decides 0, and the blue decides 1. Thus, we are violating agreement. This is the counterexample. The algorithm cannot be correct.

This proof extends to any system with n processes and $f = \frac{1}{3}$ failures, showing we need $f < \frac{1}{3}$.

Thus, we have this theorem:

Theorem. Even in a synchronous setting, there is **no algorithm solving consensus** with any-input validity (and thus all the same validity) **on authenticated channels** if one third or more of the processes are byzantine. That is **$f < N/3$** is a **necessary requirement**.

Let's see an exercise:

QUESTION: (Reasoning about Distributed Systems) An execution is "logically instantaneous" if we can consider any message in the execution as if it was instantaneously delivered without any network latency. See figure below, where, on the left, there is the actual execution and, on the right, there is the logical instantaneous equivalent execution.

Unfortunately, not all executions are logically instantaneous. See figure below, where message M3 cannot be expressed as an instantaneous message for its relationships with M1 and M2.

Discuss if the following sentence is true:

"If an execution is logically instantaneous and there are no failures, then it also respects causal order".

Discuss means doing something that is almost a proof, but probably not so rigorous.

The sentence is true.

Exercise 4:

Assume that p_0 is the writer.

② **At home**

What if the register has to be atomic?